Project 1 – Software Security Analysis

Program Development, Compilation, and Reverse Engineering Using Python, C, and IDA Pro

Introduction

This project is designed to develop, compile, and analyze programs using Python and C programming languages. As software security analysts, it is required to understand how high-level source code is translated into executable files and how these executables interact with system-level libraries. The project starts with writing and executing a simple python program in Visual Studio Code, followed by compiling the program into a Portable Executable (PE) file using PyInstaller.

Then C program was developed in Ubuntu using Windows Subsystem for Linux (WSL). This C program accepted user inputs, performed arithmetic operations, and applied conditional logic to produce different outputs. After that the program was compiled and executed using GCC compiler.

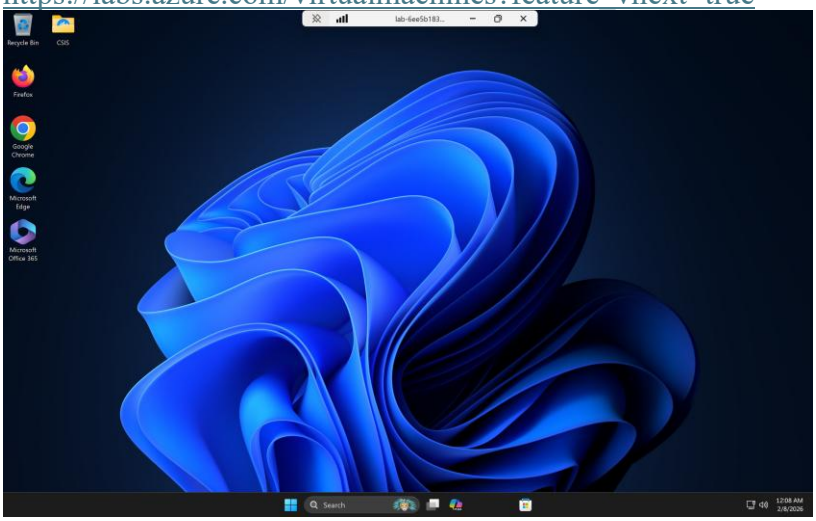
In the second part of the project started by downloading and installing IDA Pro in a Windows VM. Then both executable files were analyzed using IDA Pro to study their internal structure, imported libraries, and low-level assembly instructions. The IDA Pro helps to understand how simple programs are converted into machine level operations and how system libraries support program execution.

Part 1 - Programming and Compilation

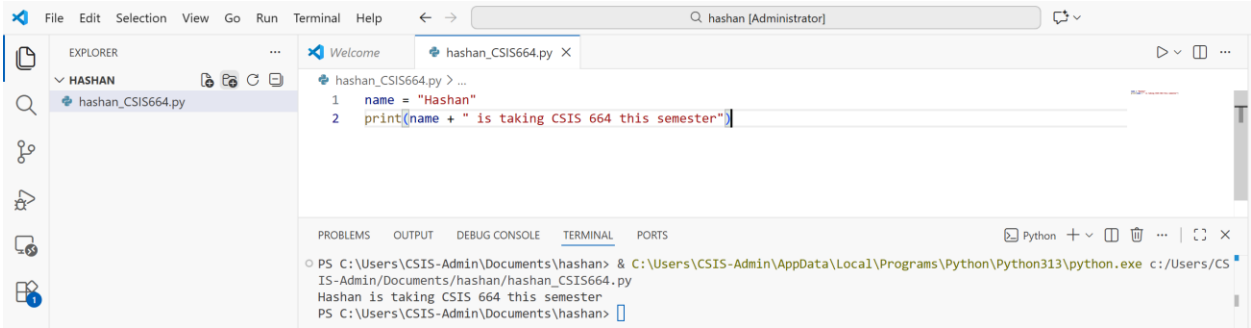
A. Create and Compilation of a Python Program into a Portable Executable (PE)

1. Logged to Azure Lab environment using following web link and logged into the Windows VM using Oracle VirtualBox.

https://labs.azure.com/virtualmachines?feature_vnext=true



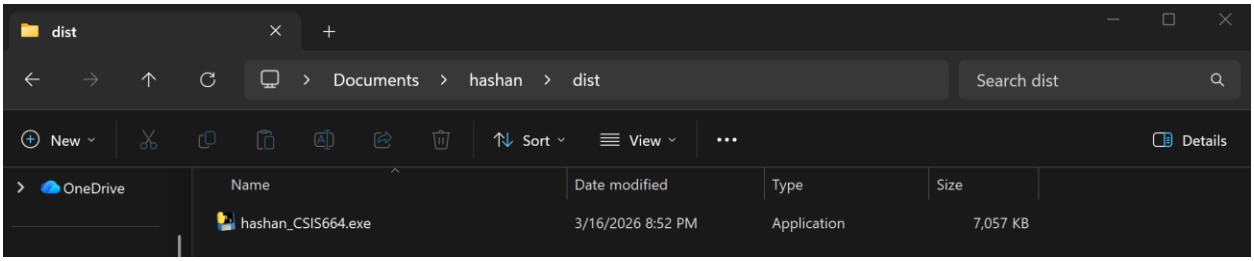
2. Created a program in the python language that gives output as *Hashan is taking CSIS 664 this semester* using VS Code.



3. The Python program compiled using *pyinstaller.* Used *--onefile hashan_CSIS664.py* command to covert file into a Portable Executable (.exe) file using PyInstaller.

```
PS C:\Users\CSIS-Admin\Documents\hashan> pyinstaller --onefile hashan_CSIS664.py
510 INFO: PyInstaller: 6.19.0, contrib hooks: 2026.1
511 INFO: Python: 3.13.1
632 INFO: Platform: Windows-11-10.0.26100-SP0
632 INFO: Python environment: C:\Users\CSIS-Admin\AppData\Local\Programs\Python\Python313
634 INFO: wrote C:\Users\CSIS-Admin\Documents\hashan\hashan_CSIS664.spec
659 INFO: Module search paths (PYTHONPATH):
['C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Scripts\\pyinstaller.exe',
 'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\python313.zip',
 'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\DLLs',
 'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Lib',
 'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313',
 'C:\\Users\\CSIS-Admin\\AppData\\Local\\Programs\\Python\\Python313\\Lib\\site-packages',
 'C:\\Users\\CSIS-Admin\\Documents\\hashan']
1268 INFO: checking Analysis
17920 INFO: Copying bootloader EXE to C:\Users\CSIS-Admin\Documents\hashan\dist\hashan_CSIS664.exe
18328 INFO: Copying icon to EXE
18531 INFO: Copying 0 resources to EXE
18532 INFO: Embedding manifest in EXE
18711 INFO: Appending PKG archive to EXE
19063 INFO: Fixing EXE headers
19558 INFO: Building EXE from EXE-00.toc completed successfully.
19560 INFO: Build complete! The results are available in: C:\Users\CSIS-Admin\Documents\hashan\dist
```

4. Navigated to the file location which was generated from previous command and confirmed the availability.



B. Design and Implementation of a C Program with Conditional Logic in Ubuntu (WSL)

1. *hashan_proj2.c* program was written in the C programming language using Ubuntu (WSL) within the Windows VM environment with following conditions.

   a. prompt the user to enter input 1 and input 2
   b. add the two inputs
   c. if the sum is less than 10, will print out the sum of the two numbers and output the string, "the sum is less than 10".
   d. if the sum is equal to or greater than 10, will print out the sum of the two numbers and output the string, "the sum is greater than (or equal) to 10".

```
hashan@lab8K4EB1:~$ nano hashan_proj2.c
```

hashan@lab8K4EB1: ~

```
  GNU nano 7.2                                  hashan_proj2.c
#include <stdio.h>

int main() {
    //Add integers
    int input1, input2, sum;

    // Prompt for user inputs
    printf("Enter input 1: ");
    scanf("%d", &input1);

    printf("Enter input 2: ");
    scanf("%d", &input2);

    // Get sum of input numbers
    sum = input1 + input2;

    // Display sum
    printf("Sum = %d\n", sum);

    // Conditions for sum
    if (sum < 10) {
        printf("The sum is less than 10\n");
    } else {
        printf("The sum is greater than (or equal) to 10\n");
    }

    return 0;
}

File Name to Write: hashan_proj2.c
^G Help              M-D DOS Format      M-A Append          M-B Backup File
^C Cancel            M-M Mac Format      M-P Prepend         ^T Browse
```

2. Executed the program and captured the results with different inputs.

```
hashan@lab8K4EB1:~$ gcc hashan_proj2.c -o hashan_proj2
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 6
Enter input 2: 8
Sum = 14
The sum is greater than (or equal) to 10
```

```
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 4
Enter input 2: 5
Sum = 9
The sum is less than 10
```

```
hashan@lab8K4EB1:~$ ./hashan_proj2
Enter input 1: 6
Enter input 2: 4
Sum = 10
The sum is greater than (or equal) to 10
```

When the sum of two integer was equal or greater than 10, it displayed the sum with a message indicating The sum is greater than (or equal) to 10. When the sum is leass than 10, it displayed the sum with a message indicating The sum is less than 10.

3. Discuss how your program in B, avoids common programming weakness.

I structured the program clearly by separating input, processing, and output. This structure improves readability and makes the program easier to understand and maintain. To have accurate calculation, I used appropriate data types, such as integers for numerical inputs.

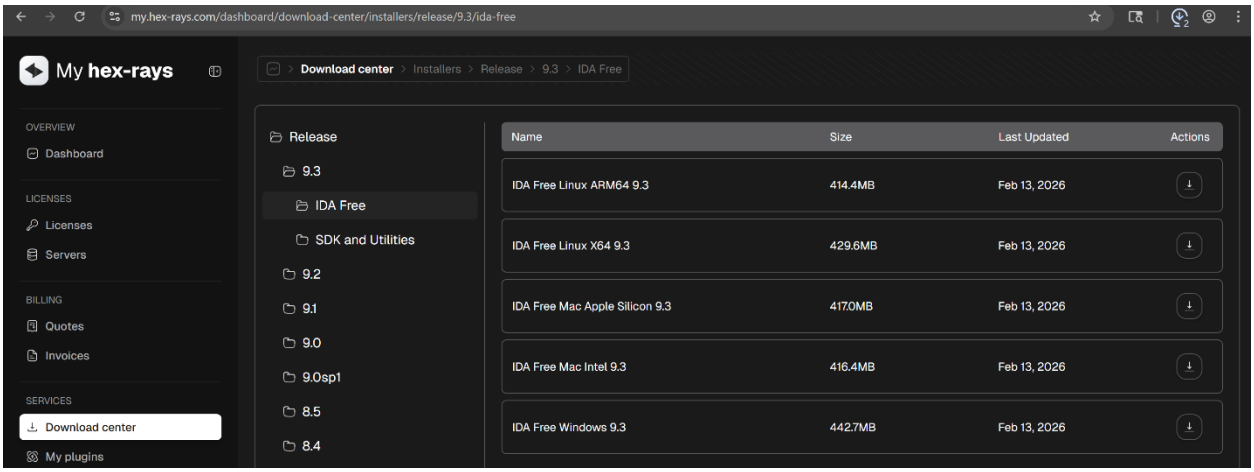
To reduce the input errors, user inputs are handled using *scanf()* function with correct format specifiers. I also used *if-else* conditional structure to control the flow of execution. This structure reduces logical errors and displays correct output messages based on computed sum of two integers.

Also, this code is easy to follow and prevents common issues like unclear logic, and poor structure. The meaningful prompts were provided to minimize the input errors, and easy to understand.

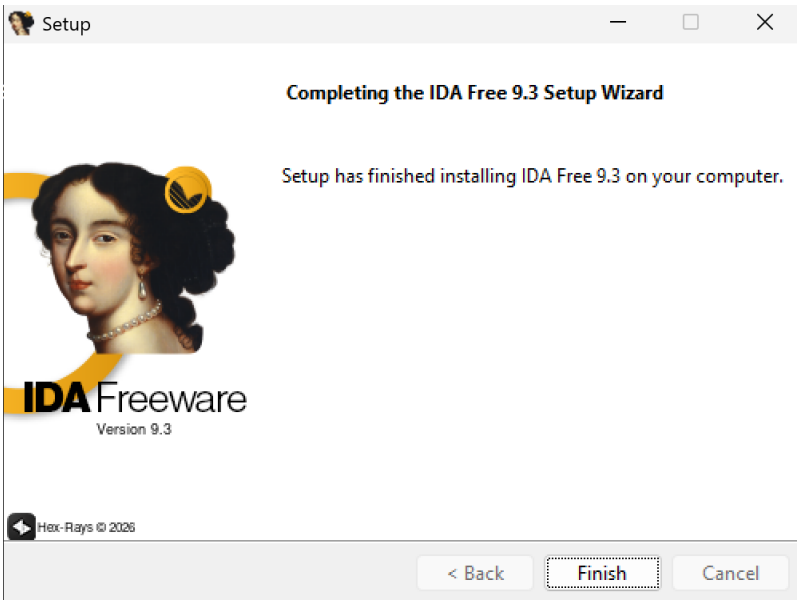
Part 2 - Reverse Engineering with IDA Pro

A. Static Analysis of a Python-Based Executable Using IDA Pro

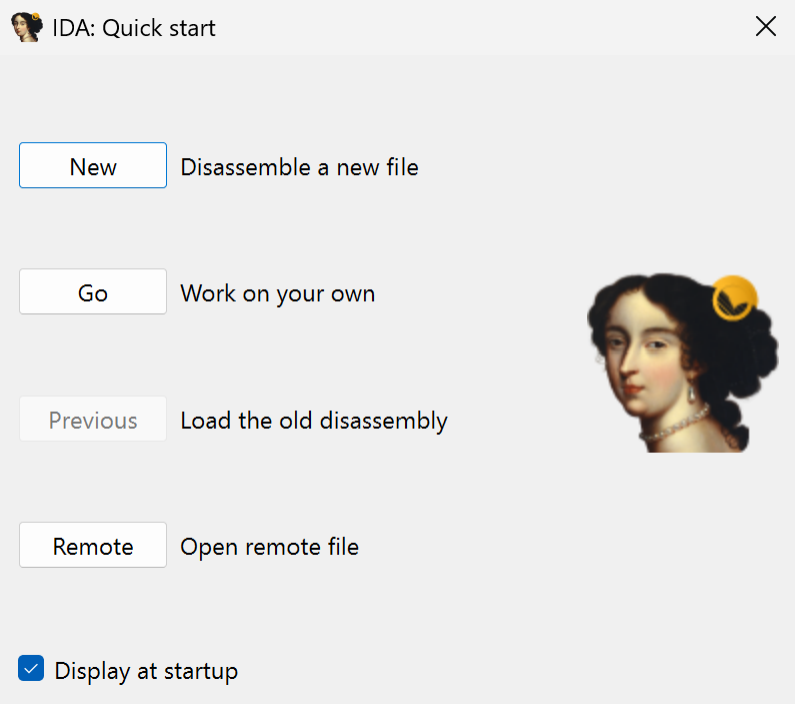
1. Downloaded a free version of IDA Pro using URL https://hex-rays.com/ida-free/#download for Windows.



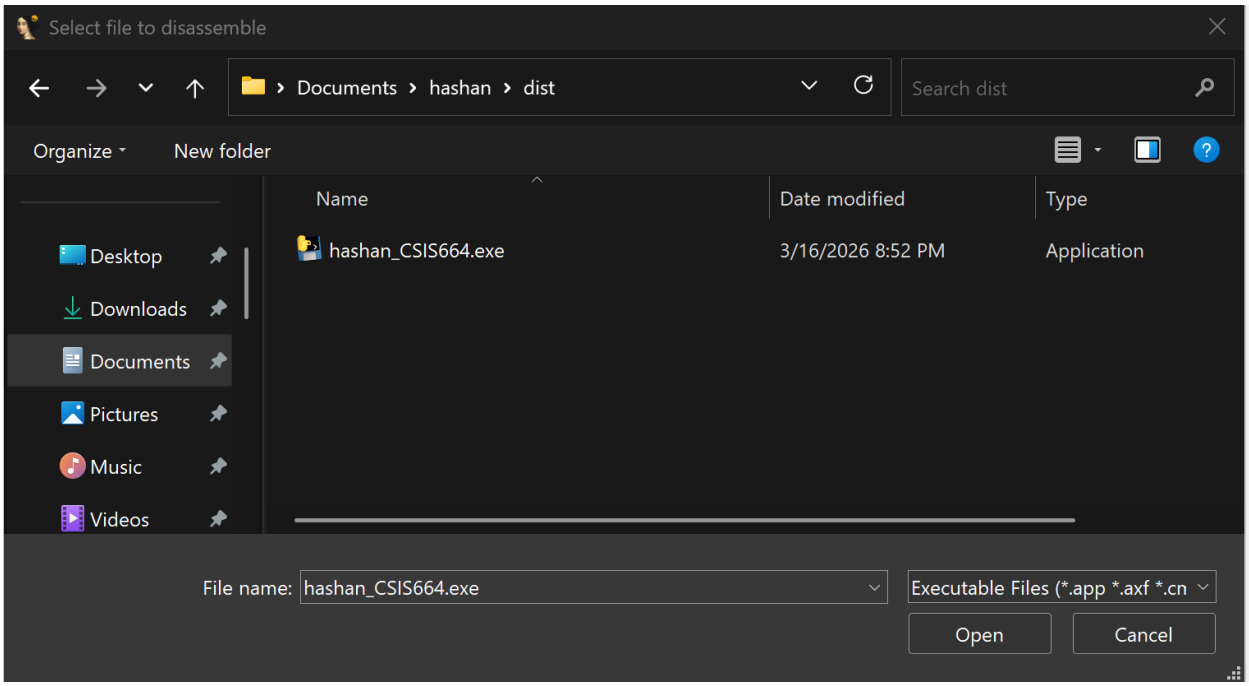
2. Installed the IDA Pro software in Windows VM.



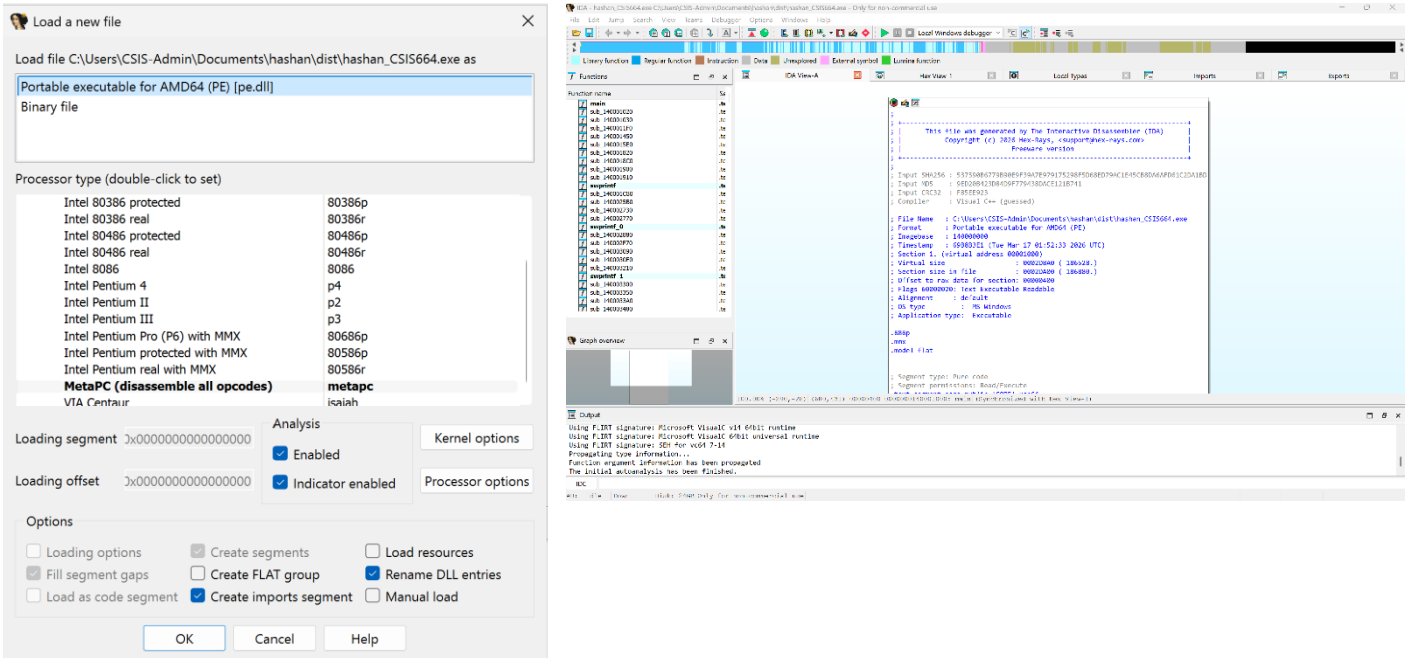
3. Started IDA Pro and selected Load new file.



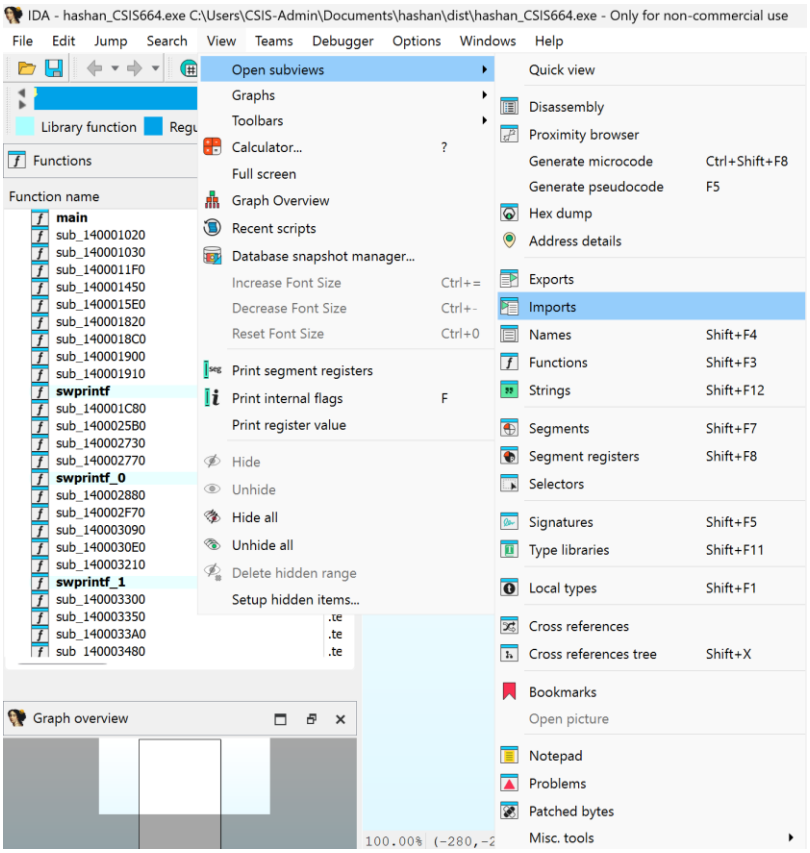
4. Navigated to *Documents/hashan/dist* to find a file which I created in Part 1 A. Then selected *hashan_CSIS664.exe* file and clicked Open.



5. In the Load new File dialog box. Selected Portable Executable (PE) format to load the.exe file. Click OK. Accept defaults in DWARF Info dialogue box.

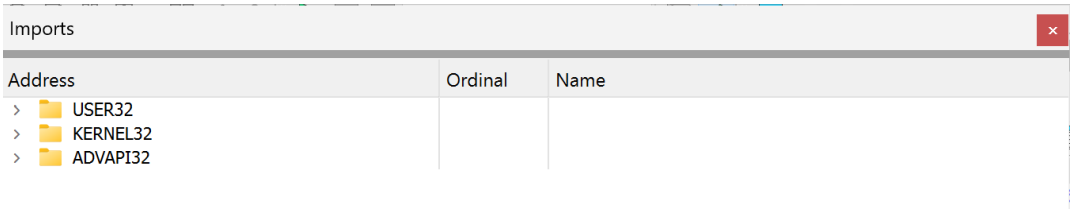


6. Navigated to the Imports view through the IDA Pro menu to examine external libraries used by the executable.



7. **What libraries are the program using?**

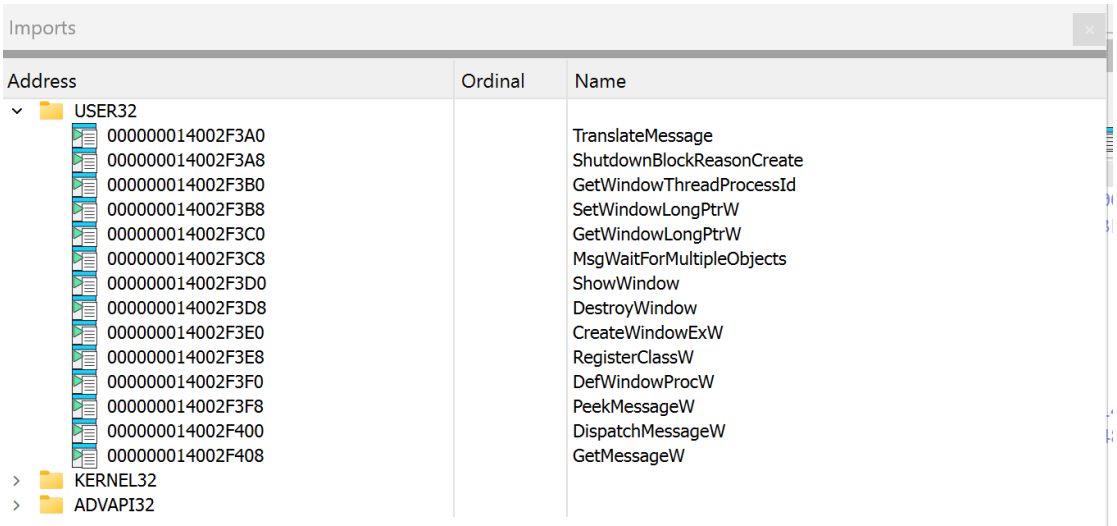
Imports window listed the external libraries used by the program, including USER32, KERNEL32, and ADVAPI32.



These are Windows API (WinAPI) libraries which provide essential functions required for the program to run in Windows environment.. These are more complex than console-based program because it needs to support GUI support and System intraction.

8. **Discuss two of these libraries and elaborate on why the program needs them.**

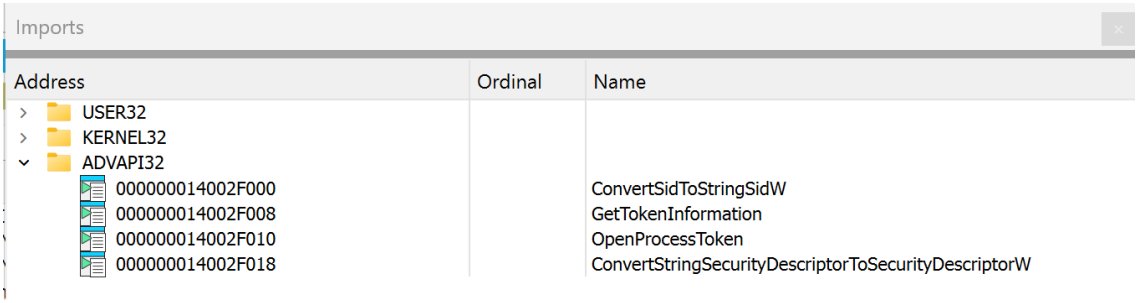
a. USER32 library expanded to display the functions related to user interface handling.



Main responsible of USER32 library includes manage user interface components and handle communication between the program and the Windows graphical system. USER32 provides window creation, message handling, and user interaction functions. In my program, it was supporting to display output and user intraction. In here, functions like ShowWindow, GetMessage, and DispatchMessage allow my program to interact with the Windows environment and respond to user actions or system events.

I wrote a simple python program to tell that “Hashan is taking CSIS 664 this semester”, still it is required USER32 library to handle interface related operation managed by the operating system. Regardless of the size or complexity of the program, USER32 library is required to enable smooth interaction between the program and the operating system’s user interface, ensuring proper handling of messages and display-related operations.

b. ADVAPI32 library expanded to display the functions related to security and system-level operations.



The KERNEL32 library is a brain of the program`s interaction with Windows operating system and it provides essential low-level system functionality required for program execution. This includes memory management, process and thread creation, file input/output operations, and system resource handling realted functions. In my program, the KERNEL32 enables the executable to interact directly with the operating system. Functions like GetModuleHandle, CreateFile, ReadFile, and ExitProcess are commonly provided by KERNEL32 library, and used to manage program execution and system-level operations.

It is required to have KERNEL32 library. Without this library, the program can not propely perform basic tasks like starting, accessing system resources, or terminating. The KERNEL32 acts as a bridge between the application and the Windows kernel. So, regardless of the size or complexity of the program, KERNEL32 library is required to ensure that instructions are executed correctly and efficiently within the system environment.

9. Looked at the IDA View-A window for my executable file.

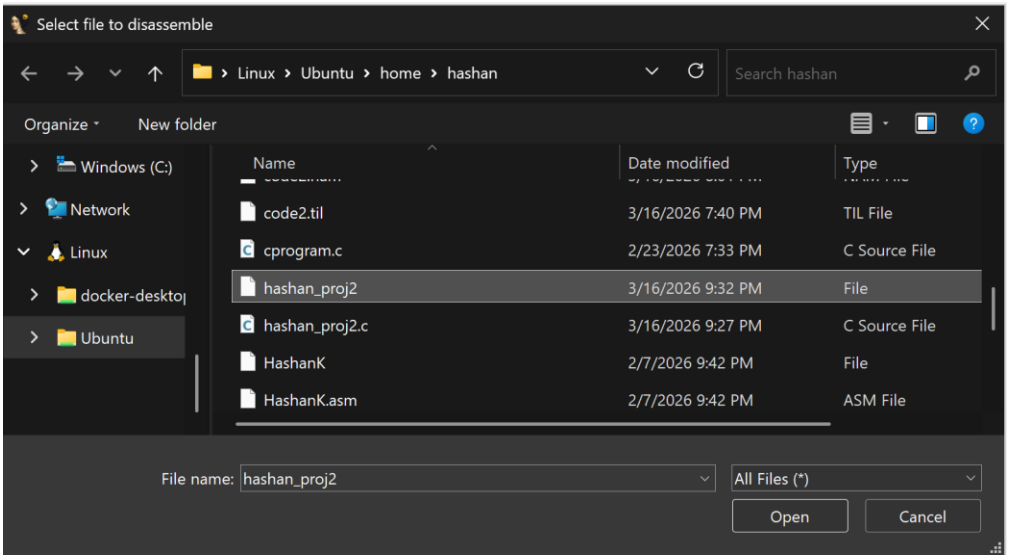
```
;
; +-------------------------------------------------------------------------+
; |      This file was generated by The Interactive Disassembler (IDA)      |
; |           Copyright (c) 2026 Hex-Rays, <support@hex-rays.com>           |
; |                            Freeware version                             |
; +-------------------------------------------------------------------------+
;
; Input SHA256 : 537590B6779B90E9F39A7E979175298F5D68ED79AC1E45CB8DA6AFD61C2DA1BD
; Input MD5    : 9ED20B423D84D9F779438DACE121B741
; Input CRC32  : F85EE923
; Compiler     : Visual C++ (guessed)

; File Name   : C:\Users\CSIS-Admin\Documents\hashan\dist\hashan_CSIS664.exe
; Format      : Portable executable for AMD64 (PE)
; Imagebase   : 140000000
; Timestamp   : 69B8B3E1 (Tue Mar 17 01:52:33 2026 UTC)
; Section 1. (virtual address 00001000)
; Virtual size                  : 0002D8A0 ( 186528.)
; Section size in file          : 0002DA00 ( 186880.)
; Offset to raw data for section: 00000400
; Flags 60000020: Text Executable Readable
; Alignment     : default
; OS type         :  MS Windows
; Application type:  Executable

.686p
.mmx
.model flat

; Segment type: Pure code
; Segment permissions: Read/Execute
_text segment para public 'CODE' use64
assume cs:_text
;org 140001000h
assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing



; int __fastcall main(int argc, const char **argv, const char **envp)
main proc near
mov     rax, cs:off_1400309C8
mov     [rax], ecx
mov     rax, cs:off_1400309C8
mov     [rax+8], rdx
mov     rcx, cs:off_1400309C8
jmp     sub_1400040E0
main endp
```

A Subroutine is a set of instructions designed to perform a specific task that can be reused multiple times within a program (GreekforGeeks, 2025). In here, *main* function is identified as the entry point of the program. Within this function, several instructions are included. First one is *mov* operation, which is used to transfer data between registers and memory locations. Then *jmp* operation redirects execution to another subroutine.

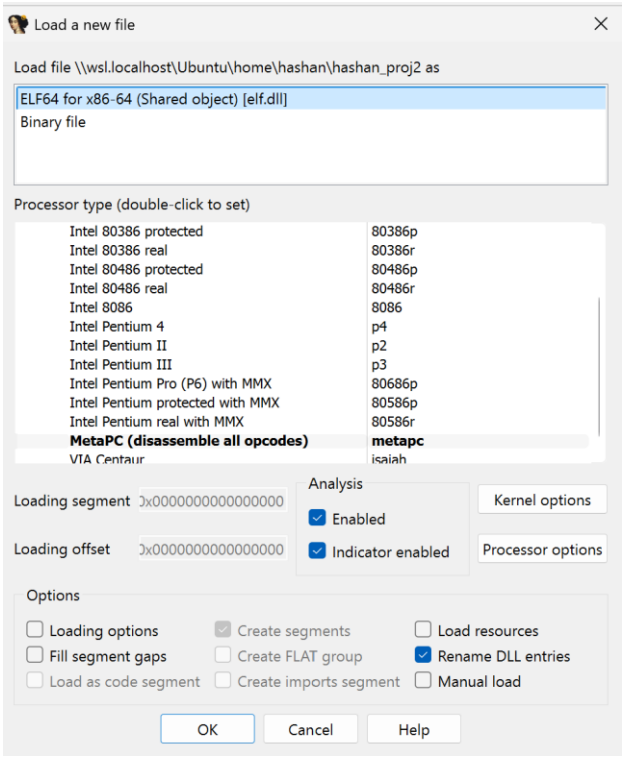
This code appears to initialize registers and prepare arguments for further processing. Values from registers like *ecx* and *rdx* are stored in memory locations referred by the *rax* register, indicating that the program is setting up parameters before calling another function. Then the jmp instructs to transfer control to another subroutine, which likely handles the main logic of the program, such as output operations.

B. Reverse Engineering of a Compiled C Program Using IDA Pro

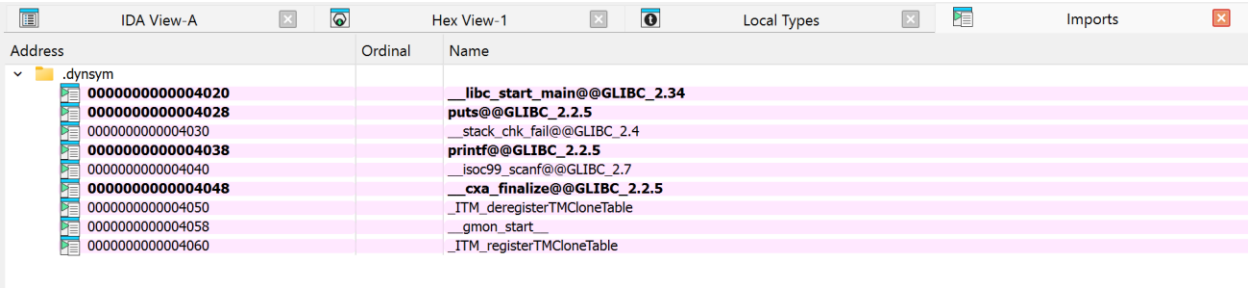
1. Navigated to *Linux/Ubuntu/home/hashan* to find a file which I created in Part 1 B. Then selected compiled C executable *hashan_proj2* file and clicked Open.



2. In the Load new File dialog box. Selected Portable Executable (PE) format to load the.exe file. Click OK. Accept defaults in DWARF Info dialogue box.



3. Displayed Imported functions of the C executable by including standard library functions.



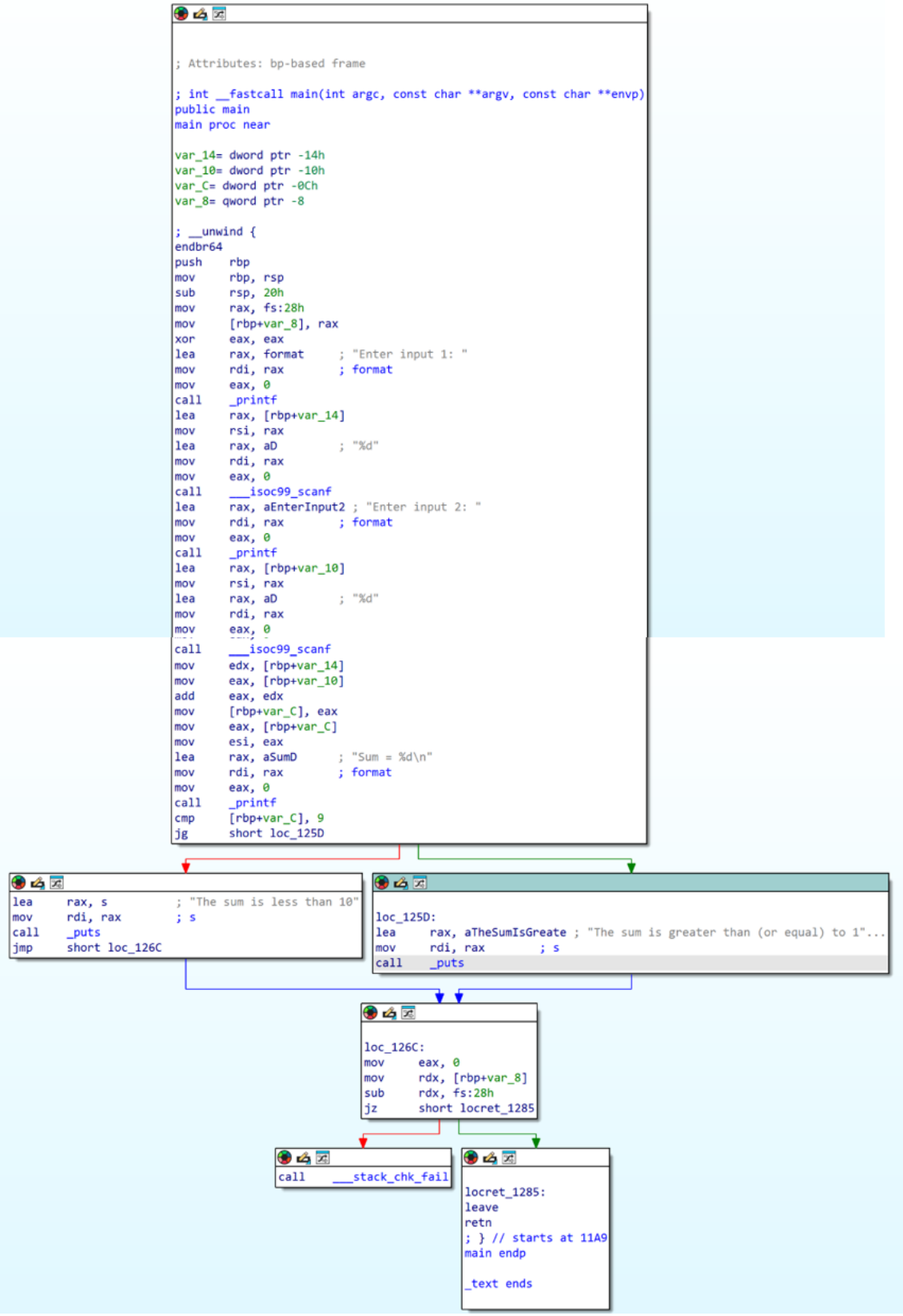
This executable uses the GNU C Library (glibc). This library is the standard C library in Linux systems. There are multiple functions like printf, scanf, puts, and __libc_start_main are listed in the Imports view, they all belong to the glibc library. Glibc library provides essential functionality required for input/output operations, program initialization, and interaction with the operating system.

The *printf* and *scanf* function are part of the glibc library and essential for input and output operations in the program. This printf function is used display output to the console, such as prompting the user and printing the calculated sum. This *scanf* function is used to receive input from the user and allow the program to read numerical values entered by the user. In my program, these functions are required to enable interaction between the user and the application.

The *__libc_start_main* function is serving as the entry point for C programs in a Linux environment. Initializing the runtime environment is the key responsible of this function. The *libc_start_main* function ensures that all necessary system-level configurations are prepared before the program begins execution.

So, without this library, the program will not start or manage execution properly. The glibc library acts as a bridge between the application and the Linux kernel to enable proper program initialization and execution.

4. Looked at the IDA View-A window for my executable file.



The IDA View-A displays how my C program operates at a low level. In here also, *main* function is identified as the entry point of the program, where the program begins execution. Within this function, several instructions are included. It starts with *push rbp* instructions and then *mov rbp, rsp,* which prepare memory for local variables. To retrieve information values and compute their sum, the *mov* and *add* instructions were used.

After getting the sum, the program compares the result with given value 10 using *cmp* instruction. Based on the result of comparison, it is directed to two branches. Each branch calls *puts* function to display the appropriate message. Then the program proceeds to finalize the program.

After displaying message, *mov eax, 0* instruction sets to return value to the program, indicating successful execution. Then the next instructions check the stack integrity check using a stack canary mechanism, which helps detect any potential memory corruption. If the check passes, the program continues to the *leave retn* normal return sequence. If the check fails, the program *calls __stack_chk_fail* function to terminate execution.

Conclusion

This project helps to cover the relationship between high-level programming and low-level execution. To observe the lower-level function, we can compile and execute Python and C programming languages in a different environment. Python scripts can be transformed into standalone executables using the Pyinstaller while C programs use GCC compilation process.

Analysis performed through IDA Pro can reveal the internal structure of executable files, including imported libraries and assembly-level instructions. Regardless of the size or complexity of the program, they intract with system libraries such as *glibc* in Linux and API libraries like *USER32, KERNEL32,* and *ADVAPI32* in Windows. It is easy to understand how functions, memory operations, and control flow are implemented at the machine level using disassembled code.

This project covers the practical knowledge of software development, compilation, and reverse engineering. It is important to know both high-level and low-level perspectives of program execution, which is essential for software security analysis.

References

1. Hex-Rays. (2024). *IDA Freeware*. https://hex-rays.com/ida-free/#download
2. OTW. (2023, November 26). *Reverse engineering malware, part 03: IDA Pro introduction*. Hackers Arise. https://hackers-arise.com/reverse-engineering-malware-part-3-ida-pro-introduction/

3. Chawdhary, G. (2025, March 23). *What are the common mistakes in software development?* Security Compass. https://securitycompass.com/blog/common-mistakes-in-software-development/

4. GeeksforGeeks. (2025, October 25). *Subroutine: Nesting and stack memory*. https://www.geeksforgeeks.org/computer-organization-architecture/subroutine-subroutine-nesting-and-stack-memory/

5. Domas, S., & Domas, C. (2024). *x86 software reverse-engineering, cracking, and counter-measures*. Wiley.

6. W3Schools. (n.d.). *C Tutorial*. https://www.w3schools.com/c/index.php